School of Information Technologies and Engineering, ADA University

CSCI4734 – Machine Learning

Fall 2025

Course Project Report

Team <12>

	Full Name	Student ID
Member 1	Nargiz Aghayeva	16042
Member 2	Raul Aghayev	17889
Member 3	Arzu Mammadova	16214

Project

A Machine Learning Model to Predict the Difficulty of Codeforces Problems

Problem formulation

Formulate the problem. What type of task is it? (50-100 words)

The task is to predict the difficulty rating of Codeforces problems based on their textual and structural features. This is a supervised regression problem, as the output (problem rating) is continuous numerical value. The input data includes problem statements, tags, problem type, and constraints. The model needs to learn patterns from these features to estimate the expected difficulty for new or unseen problems. Accurate prediction can assist in automated contest generation and personalized problem recommendations.

Discussion of related works (optional)

What has previously been done by others on this topic? (50-100 words)

Prior work on competitive programming difficulty prediction typically uses platform metadata (e.g., problem index/order, contest info, time/memory limits), behavioral signals (submission/acceptance statistics available), and content features from problem statements. Earlier approaches relied on linear models and TF-IDF style text features, while more recent work often applies gradient-boosted trees or neural text encoders (e.g., transformer embeddings) to capture semantic difficulty cues. Some studies also frame the task as ranking (learning-to-rank) rather than pure regression.

EDA and data preprocessing

Describe your EDA and data preprocessing steps. Justify your choices. Include any figures and graphs.

The dataset consists of competitive programming problems with a continuous target variable, **rating**, representing problem difficulty. As an initial step, exploratory data analysis (EDA) was conducted to understand the structure of the data, identify relevant patterns, and inform subsequent preprocessing and modeling decisions.

Target Variable Analysis

Summary statistics and visual inspection of the rating distribution revealed a wide range of values with substantial variance and a right-skewed distribution. The presence of a long tail toward higher difficulty

values indicates that extreme ratings are valid observations rather than noise. This confirmed that the task is best formulated as a **regression problem** rather than classification and motivated the use of both magnitude-based metrics (MAE, RMSE, R^2) and rank-based evaluation using **Spearman correlation**, which is particularly suitable for difficulty estimation.
Boxplot and histogram visualizations further showed no anomalous spikes or discontinuities, suggesting that outliers reflect genuinely difficult problems rather than data errors. Consequently, ratings were retained without outlier removal.

Missing Values and Data Integrity
A comprehensive missing value check across all original features, including numeric, categorical, and textual fields, confirmed that **no missing values are present** in the raw dataset. This ensured high data integrity and eliminated the need for row-level deletion or aggressive imputation strategies at the input stage. Median imputation was nevertheless applied later to engineered numeric features to ensure robustness during cross-validation and compatibility with scikit-learn pipelines.

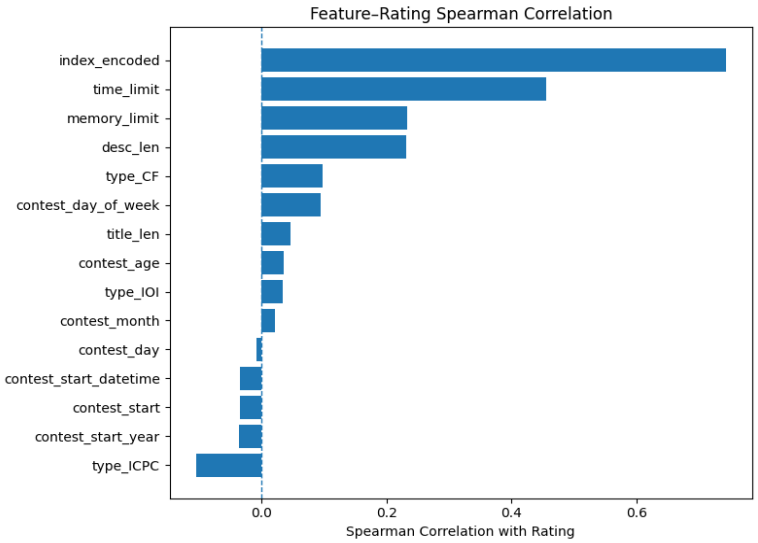
Numeric and Structural Feature Analysis
Key numeric features such as **time limit**, **memory limit**, and contest metadata were examined for scale differences and relationships with difficulty. Feature ranges varied significantly, influencing later modeling choices: linear models required scaling, whereas tree-based models did not.
Problem indices (e.g., A, B, D1) were transformed into a continuous numeric representation using a deterministic encoding that preserves ordinal structure. Spearman correlation analysis showed that this encoded index has the **strongest relationship with problem difficulty ($\rho \approx 0.74$)**, confirming that contest ordering is a strong structural signal for difficulty progression.
Time limit exhibited a moderate positive correlation with rating ($\rho \approx 0.46$), indicating that harder problems tend to allow more computational time. Memory limit showed a weaker but still positive relationship, suggesting diminishing returns beyond a certain threshold.

Text Length and Complexity Indicators
Textual fields (title, description, input format, and output format) were analyzed for length variability and formatting artifacts. Substantial variation in total text length was observed, spanning several orders of magnitude. Correlation analysis revealed weak but intuitive relationships between text length and difficulty, with description length ($\rho \approx 0.23$) and title length ($\rho \approx 0.05$) positively correlated with rating.
These findings indicate that while longer statements tend to be associated with harder problems, length alone is insufficient to capture problem complexity. This motivated the inclusion of both **simple length-based features** and **semantic text representations** in later modeling stages.

Categorical and Temporal Features
Categorical contest types were one-hot encoded to enable their use in regression models. Correlation analysis showed that contest type has limited explanatory power, with only minor positive or negative associations with difficulty. These features were retained as contextual signals rather than primary predictors.
Additional temporal features were derived from contest timestamps, including contest month, day of week, and contest age. Spearman correlation analysis indicated that these features exhibit **near-zero correlation with problem difficulty**, suggesting that difficulty is largely independent of when a contest is held. This supports the decision to treat temporal features as auxiliary information rather than dominant drivers.

Correlation Analysis

Spearman rank correlation was used to assess monotonic relationships between features and difficulty. This non-parametric approach is robust to non-linear relationships and aligns well with the ranking-oriented nature of the task. Correlation plots confirmed the dominance of structural features (problem index, time limit) and the complementary role of textual and metadata features, providing a strong empirical foundation for the selected feature set.



Data Cleaning and Feature Selection

Irrelevant or potentially leaky fields, such as contest names, tags, and example test cases, were removed to prevent unintended information leakage and reduce noise. Feature selection was guided by a combination of correlation analysis and domain knowledge: structurally meaningful features were retained, weak but intuitive signals were preserved for model learning, and temporally irrelevant attributes were downweighted or excluded from core modeling.

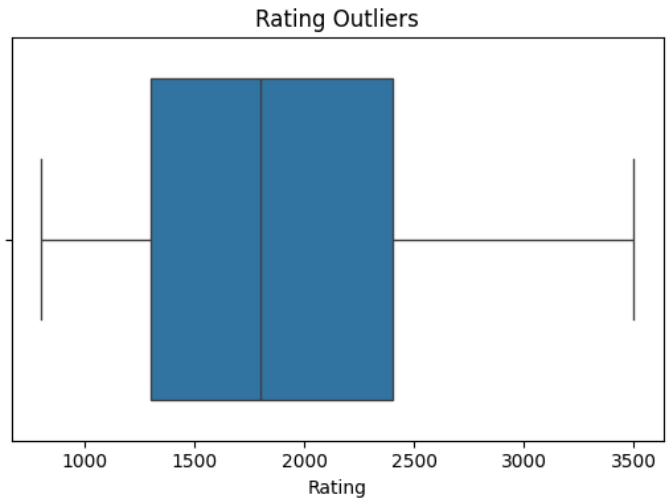
Text Preprocessing

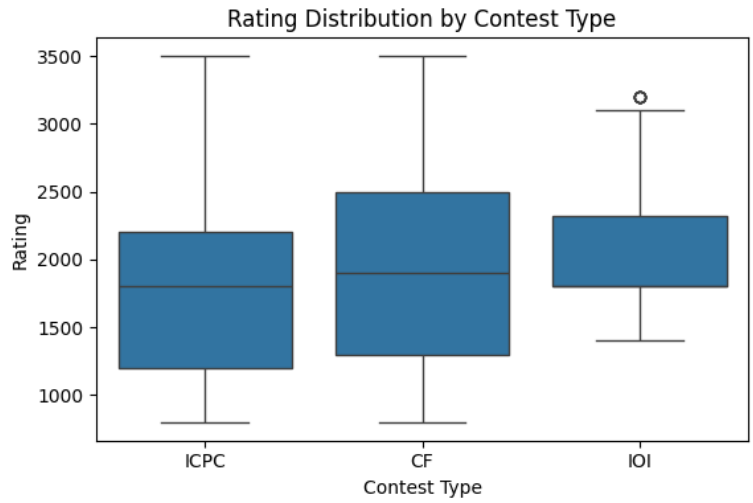
Text preprocessing was intentionally lightweight to preserve semantic content. It consisted of whitespace normalization, removal of formatting symbols, and basic cleanup of artifacts commonly found in competitive programming statements. All text fields were concatenated into a single representation and transformed using **TF-IDF vectorization with unigrams and bigrams**, providing a strong and interpretable baseline for text modeling while maintaining computational efficiency.

Pipeline-Based Preprocessing

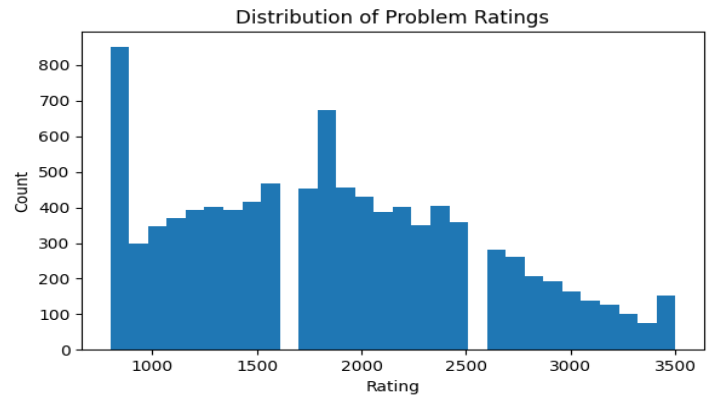
All preprocessing steps were implemented using **scikit-learn pipelines**, ensuring leakage-free cross-validation, consistent transformations across training and test sets, and full reproducibility. Separate numeric-only and text-augmented pipelines were constructed to allow fair comparison between modeling approaches and to clearly attribute performance gains to the inclusion of textual information.

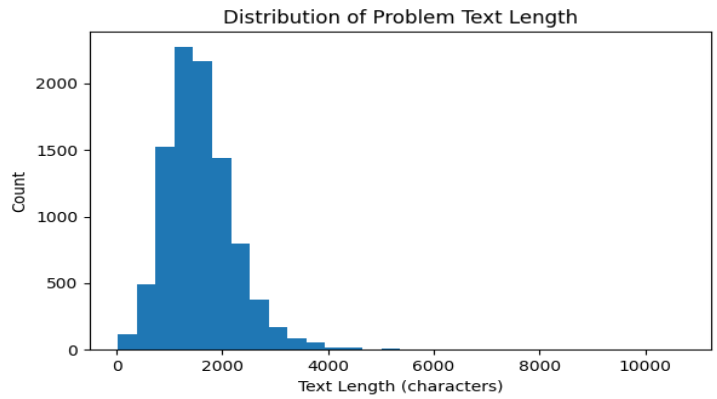
Summary

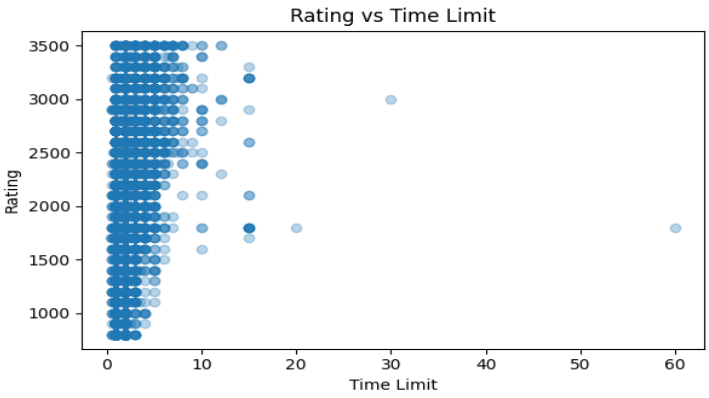


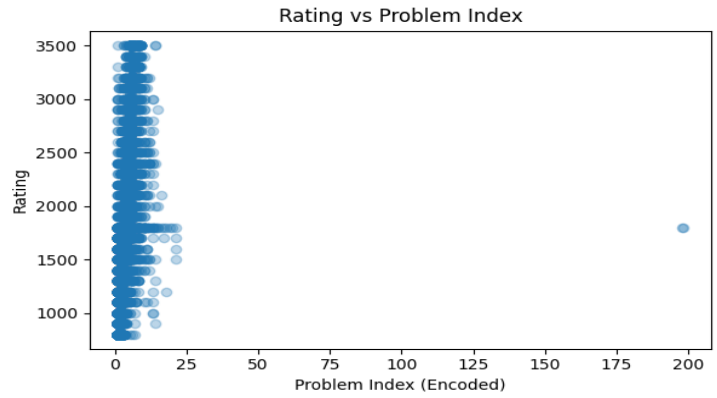


Overall, exploratory data analysis revealed that problem difficulty is primarily driven by **structural contest information and semantic content**, rather than temporal metadata. The insights gained from EDA directly informed feature engineering, preprocessing strategies, and model selection, enabling robust and interpretable modeling of competitive programming problem difficulty.









Modeling

Describe your modeling steps. Which algorithm(s) did you apply and why? What design decisions did you make? How did you evaluate your model? Have you analyzed the errors? Insert any figures.

The task was formulated as a **supervised regression problem**, where the goal is to predict a continuous difficulty rating for competitive programming problems. Multiple models were explored to capture both linear and non-linear relationships between features and problem difficulty.
The following algorithms were applied:

1. **Linear Regression and Ridge Regression**
   These models served as simple baselines. Ridge regression was included to handle

multicollinearity and stabilize coefficients through L2 regularization. These models are easy to interpret and provide a reference point for more complex methods.

2. **Support Vector Regression (LinearSVR)**
   LinearSVR was used to model linear relationships in a high-dimensional feature space, particularly when TF-IDF text features were included. Feature scaling was applied for these models, as SVR is sensitive to feature magnitude.
3. **Tree-Based Models (CatBoost and LightGBM)**
   Tree-based gradient boosting models were chosen due to their ability to:
   - capture non-linear interactions,
   - handle mixed feature types effectively,
   - perform well without feature scaling.

   CatBoost was used primarily for numeric-only experiments, while **LightGBM** was selected as the final model due to its strong performance and scalability, especially when combining textual and numeric features.

Design Decisions

Several key design decisions were made during modeling:

- **Numeric-only vs. Text + Numeric models** were evaluated separately to isolate the contribution of textual information.
- **TF-IDF vectorization** (unigrams and bigrams) was used to represent problem text efficiently without requiring large pretrained language models.
- **Pipeline-based preprocessing** was adopted to ensure that all transformations (imputation, vectorization) were applied consistently and without data leakage.
- **Median imputation** was used for numeric features due to its robustness to outliers.
- Feature scaling was applied **only where necessary** (linear models), and avoided for tree-based models.

Hyperparameters for LightGBM were tuned using cross-validation to improve generalization performance.

Model Evaluation

Model performance was evaluated using **5-fold cross-validation** on the training set. Multiple metrics were reported:

- **RMSE** to measure absolute prediction error,
- $\mathbf{R^2}$ to quantify explained variance,
- **Spearman correlation** to evaluate ranking quality, which is particularly important for difficulty estimation tasks.

Experiments and metrics were tracked using **MLflow**, enabling systematic comparison and reproducibility.

The final selected model was then retrained on the full training data and evaluated on a **held-out test set** using a leakage-free pipeline.

Error Analysis

Error analysis focused on examining prediction residuals and performance consistency across folds. Results showed that most errors occurred for problems with very high or very low difficulty ratings, where fewer training examples were available. Despite this, the model maintained strong ranking performance, indicating that relative difficulty ordering was learned effectively even when absolute errors increased.

Overall, combining numeric metadata with textual features led to modest improvements in RMSE while preserving high Spearman correlation, confirming that textual information provides complementary signals for difficulty prediction.

Experiments

What experiments did you run? Which MLOps tool(s) did you use? What are some of the advantages/disadvantages of the tool? Insert any figures.

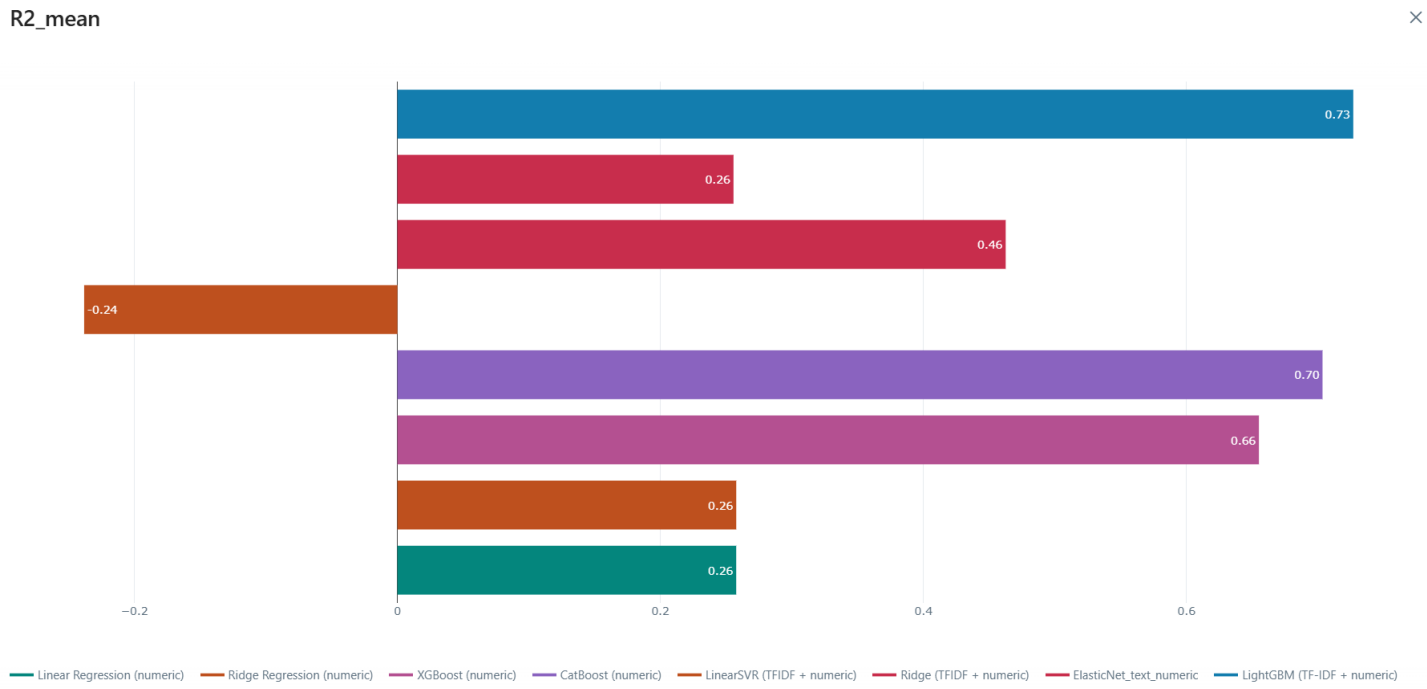
MLOps Tools Used

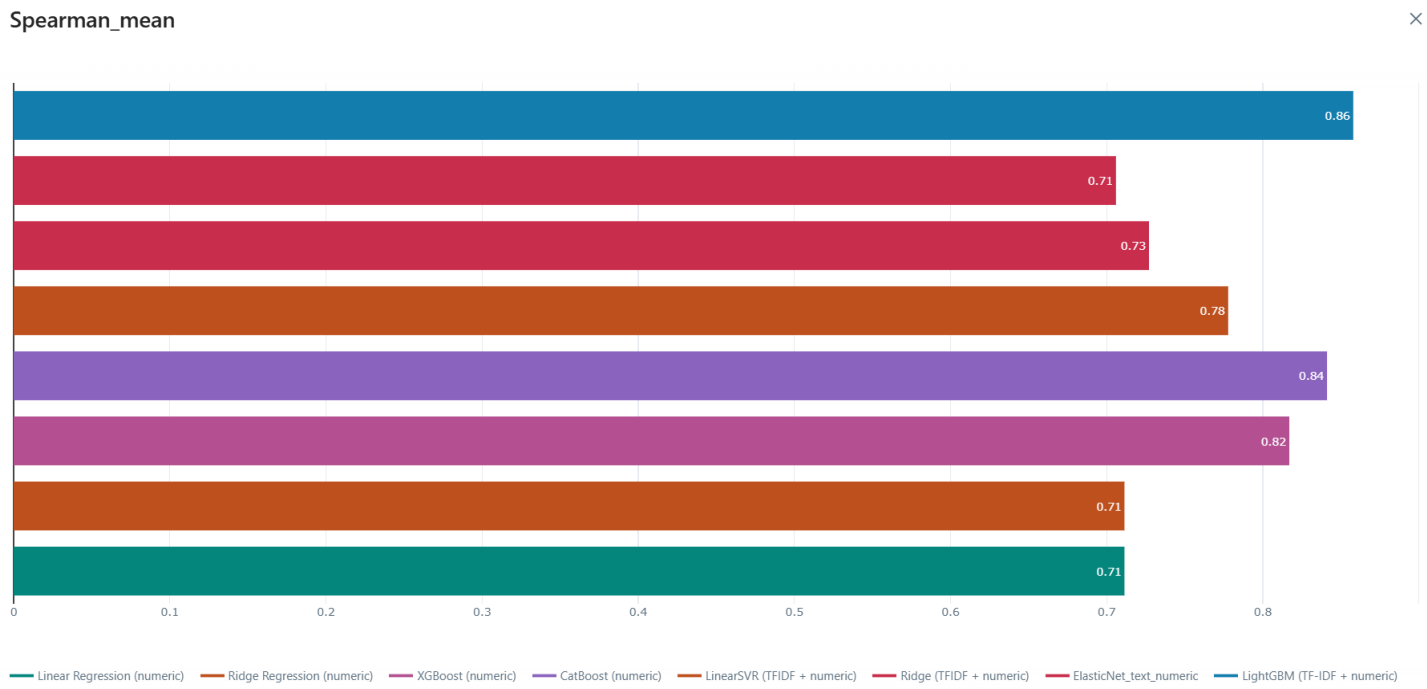
MLflow was used as the primary MLOps tool for experiment tracking and model management.
MLflow was employed to track all model runs, log hyperparameters and evaluation metrics, compare numeric-only and text-augmented experiments, store and version of the final trained model pipeline.
Each experiment (cross-validation and final test evaluation) was logged as a separate MLflow run, ensuring transparency and reproducibility.

Advantages of MLflow

MLflow provides a centralized dashboard to compare models, hyperparameters, and metrics across experiments. All preprocessing steps and trained models were saved together with their configurations, enabling reliable reloading and inference. MLflow integrates seamlessly with scikit-learn pipelines and requires minimal setup. The final model pipeline, including TF-IDF and numeric preprocessing, was stored as an artifact.

				Metrics			
Run Name	Created	Dataset	Duration	MAE_mean	R2_mean	RMSE_mean	Spearman_mear
LightGBM (TF-IDF + numeric)	7 minutes ago	-	1.8min	282.4732576...	0.726926558...	362.0624442...	0.857863060...
CatBoost (numeric)	12 minutes ago	-	6.2s	304.4935321...	0.703533868...	384.4545723...	0.841075303...
XGBoost (numeric)	13 minutes ago	-	3.9s	323.9338596...	0.655161299...	414.6496705...	0.816898143...
LinearSVR (TFIDF + numeric)	9 minutes ago	-	18.7s	378.8915178...	-0.23820378...	715.3948080...	0.777762954...
Ridge (TFIDF + numeric)	8 minutes ago	-	18.4s	388.3800940...	0.462600778...	507.2128564...	0.727046632...
Ridge Regression (numeric)	13 minutes ago	-	184ms	445.8961359...	0.257697388...	606.3413755...	0.711333282...
Linear Regression (numeric)	13 minutes ago	-	204ms	445.8867041...	0.257679839...	606.3474735...	0.711333013...
ElasticNet_text_numeric	7 minutes ago	-	23.6s	446.1142699...	0.255602489...	595.5594088...	0.705903251...





Discussion of results

How do you interpret the results of the project? Discuss the key points. (100-200 words)

The results demonstrate that both numeric metadata and textual problem descriptions are highly informative for predicting problem difficulty, but their combination yields the strongest performance. Models trained using only numeric features already achieved strong results, indicating that contest timing, problem index, and structural attributes capture much of the difficulty signal. Among numeric-only models, tree-based methods such as CatBoost and LightGBM outperformed linear models, highlighting the presence of non-linear relationships in the data.

Incorporating textual features via TF-IDF further improved performance, especially in ranking-based metrics such as Spearman correlation. This suggests that problem statements contain valuable semantic

cues related to difficulty that are not fully captured by numeric metadata alone. The tuned LightGBM model achieved the best overall performance, with high Spearman correlation and strong generalization on the held-out test set, confirming the effectiveness of both feature engineering and hyperparameter optimization.

The close alignment between cross-validation and test results indicates that the modeling pipeline is robust and free from data leakage. Overall, the experiments show that combining structured metadata with textual representations and using gradient-boosted tree models provides an effective approach for difficulty prediction.